SCOREME PLACEMENT DRIVE · HACKATHON SUBMISSION

Resilient Decision System

Configurable Workflow Decision Platform · Rajul Tiwari · github.com/billbented

GitHub repository: https://github.com/billbented/resilient-decision-system

Attached: this summary + **Solution_Report.pdf** (full architecture, design and test write-up)

85/85	4	14	17	0
tests passing	workflows, config only	explained decision examples	REST endpoints + CLI	external packages needed

What I built

One generic decision engine. Each business process is a versioned JSON workflow: **loan application** (v1 and v2), **insurance claim**, **vendor approval** and **employee onboarding**. For each request, the platform:

- validates it and blocks duplicates;
- pins it to the active rules version;
- runs it stage by stage, calling simulated external services such as a credit bureau, a document verifier and sanctions screening;
- saves progress atomically after every stage;
- ends in **Approved**, **Rejected** or **Manual review**, or waits and retries automatically if a service is down.

Every decision comes with a step-by-step explanation and a tamper-evident audit trail.

How it maps to the evaluation criteria

Criterion	Marks	Evidence
Problem Structuring	20	Five concerns, one owner each: what (JSON config) · how (pure rule engine) · when (orchestrator + state machine) · remember (store) · outside world (dependency gateway).
Architecture Quality	20	Layered: transport → application → pure domain → infrastructure. Storage and dependencies are swappable in one file (composition root).
Engineering Robustness	20	DB-enforced idempotency, atomic per-stage checkpoints, cached dependency results, leases, optimistic locking, timeouts, jittered retries, circuit breaker, scheduled workflow retries, fail-safe fallbacks.
Configurability	15	Versioned JSON workflows validated at load; atomic hot reload; in-flight requests pinned to their version; what-if simulation; one-call rollback.
Explainability & Audit	10	Every rule records actual vs expected values and the data it read; hash-chained audit verified on read; plain-language explanation endpoint.
Testing Depth	10	85 tests: concurrency race, mid-transaction failure, real timeouts, breaker recovery, audit tampering, HTTP end-to-end.
Trade-off Defence	5	Every major choice documented with the rejected alternative and why (report §12, ARCHITECTURE.md §8).

Deliverables

Required	Where (links open on GitHub)
Working runnable implementation + README	README.md · decision_platform/
Architecture document	docs/ARCHITECTURE.md

Configuration model	docs/CONFIGURATION.md · configs/*.json
API interface (REST + CLI)	api.py · __main__.py
Tests (85): happy path, invalid input, duplicates, dependency failure, retry, rule change	tests/
Decision explanation examples (14)	docs/DECISION_EXAMPLES.md · examples/explanations/
Full solution report (15 pages)	docs/Solution_Report.pdf

Constraints satisfied

Constraint	How
Tolerate requirement changes	New config version and hot reload. Example: the v2 credit threshold changed 650 → 700 with no code change, and in-flight requests stay on v1.
Simulate an external dependency	Three simulated services, with fault injection for errors, timeouts, outages and latency.
Idempotency	`Idempotency-Key` plus a payload hash, enforced by a database UNIQUE constraint. In a test, 12 concurrent duplicates produce 1 record and 1 external call.
Full audit logs	Hash-chained per request. Records every rule's values, every dependency attempt, every state change and every human decision.
Configurable without code rewrite	The employee onboarding workflow needed zero lines of code. Configs are validated at load (cycles, references, operators).
Scaling considerations	Stateless workers, leases and a queue-based async path, Postgres with `SKIP LOCKED`, audit partitioning (report §14).

Verified results

Test suite: OK, 85 tests in about 1 second. It also passes on a fresh clone of the repository.

Covers the happy path, invalid input, duplicates (including concurrent), dependency failure, retry flow, rule change, partial saves and audit tampering.

Demo outcomes (14 scenarios): 6 approved, 1 invalid, 4 manual review, 3 rejected.

Each shows the input, the rules triggered, the output and the audit reasoning.

Run it

```
git clone https://github.com/billbented/resilient-decision-system
cd resilient-decision-system
python3 -m unittest discover -s tests -t .     # 85 tests
python3 -m decision_platform demo               # 14 explained scenarios
python3 -m decision_platform serve              # REST API on :8080
```

Requires Python 3.10+ only. Implementation: 2,414 lines of platform code.